

# Spekforge

Reconstruction of API specifications  
from Documentation with LLMs

Technical Documentation of project

---

**Author:** Iván Santiago Lastra Romero

**Repository:** Spekforge

**Date:** July 20, 2026

Project - pipeline of OpenAPI specification  
reconstruction with automatic verification

# Contents

# Chapter 1

## Introduction

### 1.1 Context and motivation

Much of the API documentation is written in natural language and is available in formats designed to be read by people, not machines. This makes automation difficult, since the information is often scattered across HTML portals, PDF documents, manually exported Postman collections, or incomplete OpenAPI specifications that are no longer up to date with respect to the actual behavior of the service. In many cases, there isn't even a formal specification.

Reconstructing this information manually is a slow, repetitive process prone to human error. Therefore, automating this task is essential, as is having a tool that facilitates the generation and maintenance of up-to-date specifications.

This project automates this reconstruction process by using a local language model capable of extracting the structure of an OpenAPI specification from unstructured text documentation. However, the central aspect of the design is to strike a balance between the system's ability to extract information and its reliability, avoiding blind reliance on the responses generated by the language model. To this end, validation mechanisms are incorporated that allow for verifying the consistency and quality of the generated specifications before accepting them as the final result.

### 1.2 General objective

Build a system that functions as a tool capable of processing API documentation, regardless of its source format whether HTML, PDF, Markdown, Postman collections, or partial OpenAPI specifications and reconstruct a formal, verifiable model of the API in the form of a valid OpenAPI 3.x specification. Based on this specification, the system automatically generates three reusable artifacts, including a client SDK, a mock server, and a contract testing suite, thereby facilitating the integration, validation, and use of the API.

### 1.3 The loop of verification

Due to the phenomenon of hallucinations in language models, the process of transforming documentation into an OpenAPI 3.x specification can produce incorrect results, such as the creation of nonexistent endpoints, the erroneous assignment of data types, or the duplication of schemas. In this context, the project's goal is to implement mechanisms that detect and mitigate these inconsistencies using structural validation techniques, rule-based linting, and dynamic probing when access to the actual API is available. The discrepancies identified during these stages are used as feedback to iteratively refine the generated specification, increasing its accuracy and consistency with the service's actual behavior.

## 1.4 Scope and Approach

The project is conceived as a personal endeavor and is based on two preliminary design decisions that guide its implementation and execution.

### 1.4.1 Verification with graceful degradation

During the project’s execution, an additional dynamic probing module is developed, whose function is to compare the generated OpenAPI specification with the API’s behavior during execution, provided that a sandbox environment is available for testing. This process allows for a more representative fidelity metric by evaluating the correspondence between the reconstructed specification and the service’s actual behavior.

In cases where a sandbox environment is not available, the verification process relies exclusively on structural validation and linting mechanisms, ensuring that the specification complies with the syntactic requirements and best practices defined for OpenAPI, although without validating its conformity with the actual implementation of the API.

### 1.4.2 Target API with ground truth

An API with an official, publicly available OpenAPI specification such as those for Swagger Petstore, Stripe, or GitHub is selected as the evaluation target. During the reconstruction process, the system is provided only with the API documentation, while the official OpenAPI specification is deliberately hidden to prevent it from influencing the generated results.

Once the reconstruction is complete, the resulting specification is compared with the official specification, which serves as the ground truth. This comparison allows for the calculation of a fidelity metric that quantifies the degree of correspondence between the reconstructed specification and the original, providing an objective measure of the system’s accuracy.

## 1.5 General architecture of the system

### 1.5.1 Stages of work

The system is structured as a five-stage pipeline. An intermediate model is exchanged between each stage, this model is implemented using a set of Pydantic models that act as a data contract between the system’s various components. In this way, each stage consumes the model generated by the previous stage and produces the model for the next stage, without needing to know the internal implementation details of the other components.

1. **Ingestion and normalization** - This stage composes from the raw source and conversion to tagged fragments for each candidate endpoint
2. **LLM extraction** - Each fragment is extracted, with a structured output, that in this project is an `EndpointSpec`.
3. **Assembly** - The loose `EndpointSpec` merge in a document OpenAPI 3.x, deduplicating schemas and resolving references
4. **Verification** - Structural validation, linting, and dynamic probing.
5. **Generation** - From the verified spec it produces the SDK client, the mock server and the tests suite

### 1.5.2 The intermediate model and repository

The repository structure reflects the stages of the pipeline (`ingest/`, `extract/`, `assemble/`, `verify/`, `generate/`) plus a `models/` module that contains the shared contract among the different stages and processes of the pipeline. The goal of this is to keep each module testable in isolation.

## 1.6 Technological Stack

| Stage         | Tools                                                          |
|---------------|----------------------------------------------------------------|
| Ingest        | <code>markitdown</code> , direct JSON parsers                  |
| Extraction    | LLM with forced structured output, Chroma/Qdrant (RAG)         |
| Assembly      | Serialization Pydantic → OpenAPI 3.x                           |
| Verification  | <code>openapi-spec-validator</code> , Spectral, Schemathesis   |
| Generation    | <code>openapi-generator-cli</code> , Prism, Schemathesis/Dredd |
| Orchestration | Custom state machine                                           |

Table 1.1: Technological stack per pipeline stage

## 1.7 Document structure

This document is organized according to the project’s development phases, with each chapter corresponding to a phase; it documents the phase’s objective, the design decisions made, the relevant code, and evidence that the phase deliverable has been completed. This chapter covers exclusively **Phase 0 - Staging**; the subsequent phases (Ingest, Extraction, Assembly, Verification, and Generation) are documented in subsequent chapters as they are completed.

## Chapter 2

# Phase 0 - Staging of Project

### 2.1 Phase objective

As the first stage of development, the repository's base structure is established and the data contract is defined using the intermediate model, before implementing the system's business logic. This design decision allows all stages of the pipeline to be developed using a previously specified and consistent communication interface.

Defining the contract early on ensures that components can evolve independently, provided they adhere to the agreed-upon data model.

### 2.2 File structure

The project structure is based on packages, each of which reflects one of the project phases and can be tested individually. The following shows the organization of the repository in its main folder:

```
1 src/spekforge/  
2   models/      # shared intermediate model  
3   ingest/      # Phase 1 - Ingest and normalization  
4   extract/     # Phase 2 - LLM extraction  
5     rag/       # Module of RAG for extended documentation  
6   assemble/    # Phase 3 - Final OpenAPI assembly  
7   verify/      # Phase 4a - static and dynamic validation  
8   generate/    # Phase 4b - SDK, mock server, contract tests  
9   orchestrator/ # state proper machine  
10  evaluation/  # ground-truth metrics
```

Listing 2.1: Structure of `src/spekforge`

### 2.3 Background configuration

The project uses Python 3.12 with `uv`

#### 2.3.1 `pyproject.toml`

```
1 [project]  
2 name = "spekforge"  
3 version = "0.1.0"  
4 description = "Reconstruction of OpenAPI specifications from informal  
5   documentation, with loop verification"  
6 readme = "README.md"  
7 requires-python = ">=3.12"
```

```

7 dependencies = [
8     "pydantic>=2.7",
9     "pyyaml>=6.0",
10    "markdown>=0.0.1a3",
11    "openapi-spec-validator>=0.7",
12    "schemathesis>=3.30",
13    "httpx>=0.27",
14    "ollama>=0.3",
15    "instructor>=1.4",
16    "python-dotenv>=1.0",
17    "typer>=0.12",
18 ]
19
20 [project.optional-dependencies]
21 rag = [
22     "chromadb>=0.5",
23 ]
24
25 [dependency-groups]
26 dev = [
27     "pytest>=8.0",
28     "pytest-cov>=5.0",
29     "ruff>=0.5",
30 ]
31
32 [build-system]
33 requires = ["hatchling"]
34 build-backend = "hatchling.build"
35
36 [tool.pytest.ini_options]
37 testpaths = ["tests"]

```

Listing 2.2: Project dependencies and config

A local execution solution was chosen as the language model provider in order to avoid the costs associated with using cloud services and to ensure the tool's availability throughout the development and evaluation process. Specifically, **Ollama** was selected as the model execution environment, complemented by Instructor, a library that allows for validating generated responses against Pydantic models and performing automatic retries when the output does not conform to the expected structure.

This design decision provides a fully local, reproducible experimentation environment that is independent of external services.

### 2.3.2 .gitignore

The following are the exclusions defined for the project's version control. These include files and directories generated during development, dependencies, temporary artifacts, and other elements that are not part of the source code and are therefore excluded from the repository according to the rules set forth in the .gitignore file.

```

1 # Virtual Environment
2 .venv/
3 venv/
4
5 #uv
6 uv.lock.tmp
7
8 #Python
9 __pycache__/
10 *.py[cod]
11 *.egg-info/

```

```

12 .eggs/
13 build/
14 dist/
15
16 # testing and coverage
17 .pytest_cache/
18 .coverage
19 htmlcov/
20 .ruff_cache/
21 .mypy_cache/
22
23 # env files
24 .env
25
26 # outputs generated by pipeline
27 data/outputs/*
28 !data/outputs/.gitkeep
29
30 # Ground - truth data
31 tests/fixtures/*/ground_truth/*
32 !tests/fixtures/*/ground_truth/.gitkeep
33
34 # IDE
35 .vscode/
36 .idea/
37
38 # OS
39 .DS_Store

```

Listing 2.3: version control exclusions

For the purpose of testing, the official OpenAPI specification for the target API is deliberately excluded from the inputs provided to the system. By keeping it hidden during pipeline execution, we ensure that the reconstruction is performed solely based on the available documentation and not through direct inspection of the official specification.

This decision ensures that the comparison performed at the end of the process constitutes an objective evaluation of the system’s ability to reconstruct the specification, avoiding biases resulting from prior access to the reference document.

### 2.3.3 Dependencies installation

To install the dependencies, the command `uv sync` was executed, creating `uv.lock` and the virtual environment `.venv/`, so that the components could be tested properly. Figure 2.1 below shows the resolution and installation of dependencies without errors



```

(spekforge) ~ ivan ~/Spekforge git-[main]- uv sync
Resolved 125 packages in 4ms
Checked 89 packages in 10ms

```

Figure 2.1: Command output of `uv sync` completing dependency resolution

## 2.4 Intermediate model

To build the intermediate model, three Pydantic classes were defined, which are distributed as `ParamSpec`, `SchemaSpec`, and `EndpointSpec`, in that order of dependency.

### 2.4.1 ParamSpec

```

1 from enum import Enum
2 from pydantic import BaseModel
3
4 class ParamLocation(str, Enum):
5     PATH = "path"
6     QUERY = "query"
7     HEADER = "header"
8     COOKIE = "cookie"
9
10 class ParamSpec(BaseModel):
11     name: str
12     location: ParamLocation
13     required: bool = False
14     type: str = "string"
15     description: str | None = None
16     example: str | None = None

```

Listing 2.4: src/spekforge/models/param.py

In this case, `ParamLocation` was defined as an `Enum`, rather than a free-form text string (`str`), in order to restrict the language model, during Phase 2, to selecting only values belonging to a previously defined and valid set. When generating the JSON schema using (`model_json_schema()`), this enumeration translates into an explicit restriction within the *JSON Schema* used during the *function calling*.

### 2.4.2 SchemaSpec

```

1 from pydantic import BaseModel, Field
2
3 class SchemaSpec(BaseModel):
4     name: str | None = None
5     type: str = "object"
6     format: str | None = None
7     description: str | None = None
8     properties: dict[str, "SchemaSpec"] = Field(default_factory=dict)
9     required: list[str] = Field(default_factory=list)
10    items: "SchemaSpec | None" = None
11    enum: list[str] | None = None
12    ref: str | None = None
13
14 SchemaSpec.model_rebuild()

```

Listing 2.5: src/spekforge/models/schema.py

The `SchemaSpec` model has a recursive structure, the `properties` attribute corresponds to a set of objects of the same `SchemaSpec` type, while the `items` field represents the schema of the elements contained in an array. This definition allows for the modeling of nested data structures, such as a `Pet` object that contains a list of `Tag` objects, while preserving the hierarchy of the OpenAPI specification.

Due to this self-reference, it is necessary to call the `model_rebuild()` method once the class definition is complete. This procedure resolves references to the class itself, since, during the evaluation of the `SchemaSpec` body, the corresponding symbol has not yet been fully defined.

The `ref` field is intended for Phase 3; `assemble/dedup.py` will use it to indicate that a schema has already been registered in the components under the specified name. This is done to prevent the LLM from describing the same object in slightly different ways in different sections of the documentation.

### 2.4.3 EndpointSpec

```

1 from enum import Enum
2 from pydantic import BaseModel, Field
3 from spekforge.models.schema import SchemaSpec
4 from spekforge.models.param import ParamSpec
5
6 class HTTPMethod(str, Enum):
7     GET= "get"
8     POST= "post"
9     PUT= "put"
10    DELETE= "delete"
11    HEAD= "head"
12    OPTIONS= "options"
13    PATCH= "patch"
14
15 class AuthType(str, Enum):
16     NONE= "none"
17     API_KEY="api_key"
18     BEARER="bearer"
19     BASIC="basic"
20     OAUTH2="oauth2"
21
22 class EndpointSpec(BaseModel):
23     method: HTTPMethod
24     path: str
25     summary: str | None = None
26     description: str | None = None
27     parameters: list[ParamSpec] = Field(default_factory=list)
28     request_body: SchemaSpec | None = None
29     responses: dict[str, SchemaSpec] = Field(default_factory=dict)
30     auth: AuthType = AuthType.NONE
31     errors: list[str] = Field(default_factory=list)

```

Listing 2.6: src/spekforge/models/endpoint.py

EndpointSpec is the central component of the intermediate model, so Phase 2 is responsible for populating the model for each endpoint detected in the documentation, and Phase 3 uses this to build the final model with the specified configuration. The **responses** field is modeled directly as an extension of `dict[str, SchemaSpec]`, with the HTTP status code as the key, mapping directly to the response block in the specification. Both **request\_body** and **auth** are optional modules that are populated since not all endpoints have explicit authentication or a request body.

## 2.5 Deliverable Validation

### 2.5.1 Tests

```

1 from spekforge.models.endpoint import EndpointSpec, HTTPMethod, AuthType
2 from spekforge.models.param import ParamSpec, ParamLocation
3 from spekforge.models.schema import SchemaSpec
4
5 def test_endpoint_spec_minimal():
6     endpoint = EndpointSpec(method=HTTPMethod.GET, path="/pets/{id}")
7
8     assert endpoint.method == HTTPMethod.GET
9     assert endpoint.path == "/pets/{id}"
10    assert endpoint.auth == AuthType.NONE
11    assert endpoint.parameters == []
12

```

```

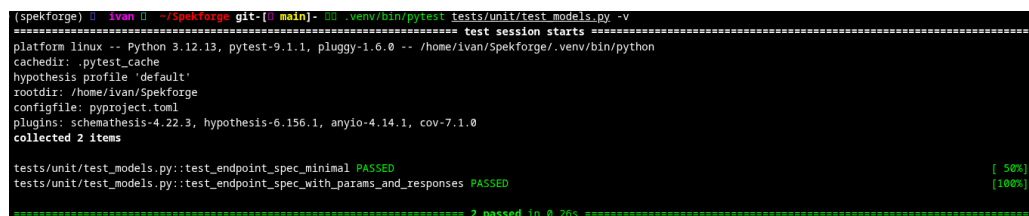
13 def test_endpoint_spec_with_params_and_responses():
14     pet_schema = SchemaSpec(
15         name="Pet",
16         properties={
17             "id": SchemaSpec(type="integer"),
18             "name": SchemaSpec(type="string"),
19         },
20         required=["id", "name"],
21     )
22     id_param = ParamSpec(name="id", location=ParamLocation.PATH, required=True,
23                          type="integer")
24
25     endpoint = EndpointSpec(
26         method=HTTPMethod.GET,
27         path="/pets/{id}",
28         parameters=[id_param],
29         responses={"200": pet_schema},
30     )
31
32     assert endpoint.parameters[0].name == "id"
33     assert endpoint.responses["200"].properties["name"].type == "string"

```

Listing 2.7: tests/unit/test\_models.py

The first test covers a minimum case required by the phase deliverable. This test verifies the functionality of the intermediate model; in addition, a second test is performed to verify the nesting of ParamSpec and SchemaSpec within EndpointSpec, including the recursion of SchemaSpec.properties.

## 2.5.2 Execution



```

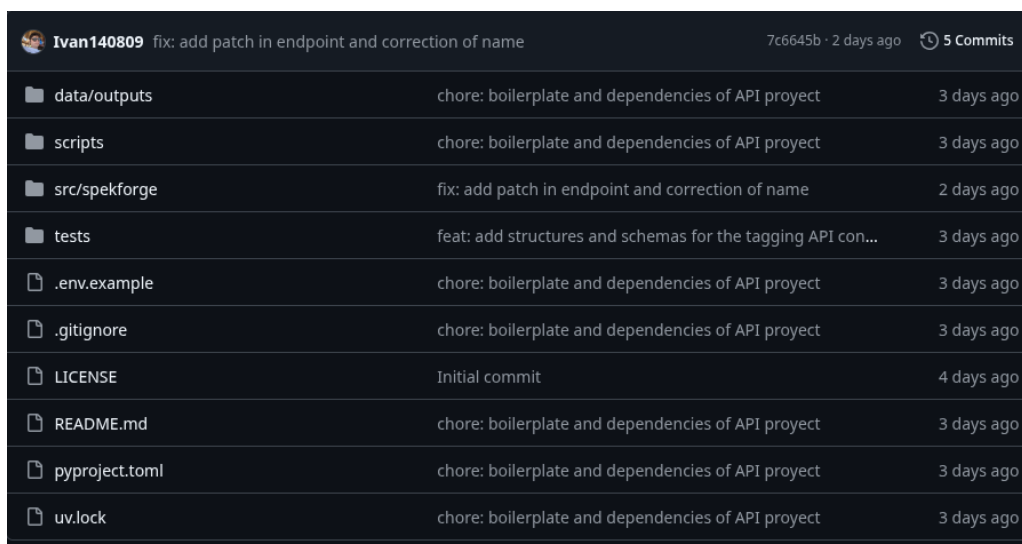
(spekforge) U Ivan U ~/Spekforge git-[U main]- U .venv/bin/pytest tests/unit/test_models.py -v
===== test session starts =====
platform linux -- Python 3.12.13, pytest-9.1.1, pluggy-1.6.0 -- /home/ivan/Spekforge/.venv/bin/python
cachedir: .pytest_cache
hypothesis profile 'default'
rootdir: /home/ivan/Spekforge
configfile: pyproject.toml
plugins: schemathesis-4.22.3, hypothesis-6.156.1, anyio-4.14.1, cov-7.1.0
collected 2 items

tests/unit/test_models.py::test_endpoint_spec_minimal PASSED [ 50%]
tests/unit/test_models.py::test_endpoint_spec_with_params_and_responses PASSED [100%]

===== 2 passed in 0.26s =====

```

Figure 2.2: pytest output showing the two passing tests of the intermediate model



|                                                                                                     |                                                             |                      |                                                                                               |
|-----------------------------------------------------------------------------------------------------|-------------------------------------------------------------|----------------------|-----------------------------------------------------------------------------------------------|
|  <b>Ivan140809</b> | fix: add patch in endpoint and correction of name           | 7c6645b · 2 days ago |  5 Commits |
| data/outputs                                                                                        | chore: boilerplate and dependencies of API proyeect         | 3 days ago           |                                                                                               |
| scripts                                                                                             | chore: boilerplate and dependencies of API proyeect         | 3 days ago           |                                                                                               |
| src/spekforge                                                                                       | fix: add patch in endpoint and correction of name           | 2 days ago           |                                                                                               |
| tests                                                                                               | feat: add structures and schemas for the tagging API con... | 3 days ago           |                                                                                               |
| .env.example                                                                                        | chore: boilerplate and dependencies of API proyeect         | 3 days ago           |                                                                                               |
| .gitignore                                                                                          | chore: boilerplate and dependencies of API proyeect         | 3 days ago           |                                                                                               |
| LICENSE                                                                                             | Initial commit                                              | 4 days ago           |                                                                                               |
| README.md                                                                                           | chore: boilerplate and dependencies of API proyeect         | 3 days ago           |                                                                                               |
| pyproject.toml                                                                                      | chore: boilerplate and dependencies of API proyeect         | 3 days ago           |                                                                                               |
| uv.lock                                                                                             | chore: boilerplate and dependencies of API proyeect         | 3 days ago           |                                                                                               |

Figure 2.3: Repository file tree on GitHub, showing the complete structure of `src/spekforge/` at the close of Phase 0

## Chapter 3

# Phase 1 - Ingestion

### 3.1 Phase objective

Convert a raw documentation source into a list of fragments tagged by candidate endpoint, regardless of whether that source is unstructured prose like PDF, HTML, or Markdown, or already structured data like a partial OpenAPI document, a Postman collection, or a HAR capture. This phase produces the input that Phase 2 will consume, and defines the second data contract of the pipeline: the `Chunk` model.

### 3.2 The Chunk model

```
1 from enum import Enum
2 from pydantic import BaseModel
3 from spekforge.models.endpoint import EndpointSpec
4
5 class SourceType(str, Enum):
6     MARKDOWN = "markdown"
7     OPENAPI_PARTIAL = "openapi_partial"
8     POSTMAN = "postman"
9     HAR = "har"
10
11 class Chunk(BaseModel):
12     id: str
13     source_type: SourceType
14     source_path: str
15     raw_text: str
16     candidate_method: str | None = None
17     candidate_path: str | None = None
18     pre_extracted: EndpointSpec | None = None
```

Listing 3.1: `src/spekforge/models/chunk.py`

`Chunk` lives in `models/`, alongside `EndpointSpec`, `ParamSpec`, and `SchemaSpec`, because it is a contract between two stages of the pipeline (Ingestion  $\rightarrow$  Extraction), following the same pattern established in Phase 0.

The fields `candidate_method` and `candidate_path` are typed as plain `str | None`, not as the strict `HTTPMethod` enum used inside `EndpointSpec`: at this stage they are only a heuristic guess extracted from a heading. This avoids forcing the enum too early, which would break ingestion the moment a heading did not match the expected pattern exactly.

The `pre_extracted` field is the mechanism that allows already structured sources to skip the LLM call in Phase 2: when this field is populated, the orchestrator uses it directly; when it is `None`, `raw_text` is sent to the model instead. This keeps `to_intermediate()` returning a uniform `list[Chunk]` regardless of which ingestor produced it.



This design carries a known limitation: if a documentation source does not follow this heading convention, `split_into_chunks` returns an empty list. No fallback has been implemented yet; the idea for a future expansion is to build one, but only once there is real data to validate what a sensible fallback should look like.

### 3.5 MarkdownIngestor

```

1 from pathlib import Path
2 from markdown import Markdown
3 from spekforge.ingest.chunking import split_into_chunks
4 from spekforge.models.chunk import Chunk
5
6 _SUPPORTED_EXTENSIONS = {".pdf", ".docx", ".pptx", ".html", ".htm", ".md"}
7
8 class MarkdownIngestor:
9     def __init__(self) -> None:
10         self.converter = Markdown()
11
12     def accepts(self, source: Path) -> bool:
13         return source.suffix.lower() in _SUPPORTED_EXTENSIONS
14
15     def to_intermediate(self, source: Path) -> list[Chunk]:
16         result = self.converter.convert(str(source))
17         return split_into_chunks(result.markdown, str(source))

```

Listing 3.4: `src/spekforge/ingest/markdown_ingestor.py`

This ingestor accepts the set of extensions listed in `_SUPPORTED_EXTENSIONS`, converts the source to Markdown through `markdown`, and delegates the boundary-detection logic to `chunking.py`. Extension recognition was verified for all six supported extensions, and the full conversion path was additionally validated end to end against real `.html` and `.md` fixtures, confirming that raw input is correctly converted and boundary-split into chunks; `.pdf`, `.docx`, and `.pptx` were checked only for extension acceptance so far.

### 3.6 StructuredIngestor

This ingestor handles three distinct already-structured formats and converts each one directly into `EndpointSpec` objects, bypassing the LLM entirely for that data.

#### 3.6.1 Format detection

```

1 def _detect_format(data: dict) -> SourceType | None:
2     if not isinstance(data, dict):
3         return None
4     if "paths" in data and ("openapi" in data or "swagger" in data):
5         return SourceType.OPENAPI_PARTIAL
6     if "item" in data and "info" in data:
7         return SourceType.POSTMAN
8     if "entries" in data.get("log", {}):
9         return SourceType.HAR
10    return None

```

Listing 3.5: Format detection heuristic

Detection relies on top-level JSON keys, because all three formats are plain `.json` (or, in the case of HAR, sometimes `.har`) and are only distinguishable by their shape.

### 3.6.2 OpenAPI partial

```

1 def _from_openapi(data: dict) -> list[tuple[EndpointSpec, dict]]:
2     results = []
3     for path, path_item in data.get("paths", {}).items():
4         for method_str, operation in path_item.items():
5             if method_str.upper() not in HTTPMethod.__members__:
6                 continue
7
8             endpoint = EndpointSpec(
9                 method=HTTPMethod(method_str.lower()),
10                path=path,
11                summary=operation.get("summary"),
12                description=operation.get("description"),
13                parameters=[_openapi_param(p) for p in
14                    operation.get("parameters", [])],
15                request_body=_openapi_request_body(operation.get("requestBody")),
16                responses=_openapi_responses(operation.get("responses", {})),
17            )
18            results.append((endpoint, {path: {method_str: operation}}))
19 return results

```

Listing 3.6: Converting OpenAPI into an EndpointSpec

A recursive helper, `_openapi_schema`, translates nested JSON Schema objects (`properties`, `items`, `required`) into `SchemaSpec` instances, mirroring the recursive structure already defined in Phase 0. If a response is declared without a `content` block, such as a 204, it is correctly omitted from `responses` instead of being assigned an invented schema.

### 3.6.3 Postman collections

Postman collections nest requests inside folders arbitrarily deep, so traversal is recursive, as shown below:

```

1 def _walk_postman_items(items: list, results: list[tuple[EndpointSpec, dict]])
2     -> None:
3     for item in items:
4         if "item" in item:
5             _walk_postman_items(item["item"], results)
6             continue
7         if "request" not in item:
8             continue
9         request = item.get("request", {})
10        method_str = request.get("method", "GET").upper()
11        if method_str not in HTTPMethod.__members__:
12            continue
13
14        url = request.get("url", "")
15        endpoint = EndpointSpec(
16            method=HTTPMethod(method_str.lower()),
17            path=_postman_path(url),
18            summary=item.get("name"),
19            parameters=_postman_params(request, url),
20        )
21        results.append((endpoint, item))
22
23 def _postman_path(url) -> str:
24     if isinstance(url, str):
25         path = urlparse(url).path
26     else:
27         path = "/" + "/".join(url.get("path", []))
28     segments = [f"{{{p[1:]}}}" if p.startswith(":") else p for p in
29         path.split("/")]

```

```

28     return "/" . join(segments) or "/"
29
30 def _postman_params(request: dict, url) -> list[ParamSpec]:
31     params = [
32         ParamSpec(name=h["key"], location=ParamLocation.HEADER, required=not
33             h.get("disabled", False), type="string")
34         for h in request.get("header", [])
35         if h.get("key") and not h.get("disabled", False)
36     ]
37     if isinstance(url, dict):
38         params += [
39             ParamSpec(name=q["key"], location=ParamLocation.QUERY, required=not
40                 q.get("disabled", False), type="string")
41             for q in url.get("query", [])
42             if q.get("key") and not q.get("disabled", False)
43         ]
44     return params

```

Listing 3.7: Recursive traversal, with validation of malformed items

Postman's path-variable syntax is normalized to the OpenAPI style, so that `EndpointSpec.path` follows a single convention regardless of the originating format. Postman collections do not carry real response schemas, so that field is intentionally left empty for this format. Item traversal was validated against malformed items and against header/query parameters explicitly marked as disabled in the collection.

### 3.6.4 HAR archives

HAR is the only one of the three formats that carries a real, captured response body, which makes it the most reliable source for inferring response schemas from actual data rather than declared types:

```

1 def _value_to_schema(value) -> SchemaSpec:
2     if isinstance(value, dict):
3         return SchemaSpec(
4             type="object",
5             properties={k: _value_to_schema(v) for k, v in value.items()},
6             required=list(value.keys()),
7         )
8     if isinstance(value, list):
9         return SchemaSpec(type="array", items=_value_to_schema(value[0]) if
10             value else None)
11     if isinstance(value, bool):
12         return SchemaSpec(type="boolean")
13     if isinstance(value, int):
14         return SchemaSpec(type="integer")
15     if isinstance(value, float):
16         return SchemaSpec(type="number")
17     if value is None:
18         return SchemaSpec(type="null")
19     return SchemaSpec(type="string")

```

Listing 3.8: Inferring a schema from a captured JSON value

This function checks the type of the captured value in sequence, starting with `bool` before `int` so that a JSON boolean is not misclassified as an integer, and returns the corresponding `SchemaSpec`. It was also verified that an entry declaring an invalid HTTP method is skipped rather than processed.

## 3.7 Router

```
1 from pathlib import Path
2 from spekforge.ingest.base import Ingestor
3 from spekforge.ingest.markitdown_ingestor import MarkdownIngestor
4 from spekforge.ingest.structured_ingestor import StructuredIngestor
5 from spekforge.models.chunk import Chunk
6
7 _INGESTORS: list[Ingestor] = [StructuredIngestor(), MarkdownIngestor()]
8
9 def route(source: Path) -> list[Chunk]:
10     for ingestor in _INGESTORS:
11         if ingestor.accepts(source):
12             return ingestor.to_intermediate(source)
13
14     raise ValueError(f"No ingestor accepts source: {source}")
```

Listing 3.9: src/spekforge/ingest/router.py

`route()` is in charge of selecting the ingestor depending on the source. `StructuredIngestor` is tried first, since it is the more specific of the two, validating the actual JSON content shape rather than just the extension. If no registered ingestor accepts the source, the function raises a `ValueError` including the source path. This also covers a source path that does not exist on disk: `accepts()` catches the `OSError` raised while attempting to read it and simply returns `False`, so a missing file surfaces as the same explicit error as any other unrecognized source.

## 3.8 Validation against real documentation

To validate the pipeline against something more realistic than hand-written fixtures, the official OpenAPI 3.0 specification for Swagger Petstore was retrieved directly from its public instance ([petstore3.swagger.io](https://petstore3.swagger.io)) and stored as the hidden ground truth for this project, in `tests/fixtures/petstore/ground_truth/`.

The Swagger UI documentation portal itself, however, could not be used directly as an ingestion source: it is a single-page application whose static HTML contains nothing but an empty `div` placeholder, with all real content rendered client-side by JavaScript. To still validate the pipeline against realistic narrative documentation, a Markdown document was generated directly from the official specification's 19 operations, following the same heading-per-endpoint convention real documentation generators produce, and stored separately in `tests/fixtures/petstore/input_docs/`, apart from the hidden ground truth.

Running the full pipeline (`router.route()` → `MarkdownIngestor` → `chunking.py`) against this generated Markdown fixture detected all 19 operations defined in the official specification, with zero missed and zero spurious endpoints. The next figure shows this ingestion run.

### 3.8.1 Edge cases review

Beyond the fixture above, every component of this phase was additionally exercised against edge cases designed to probe the phase further, such as empty input; boundary conditions in heading detection like consecutive headings, duplicate headings, and unusual whitespace; invalid methods; extension case sensitivity; malformed and non-existing source files; OpenAPI responses without a body; and invalid HTTP methods in Postman and HAR entries. This review is what led to the guard clauses already shown in this chapter's code, such as the malformed-item check in `_walk_postman_items` and the `disabled` filter in `_postman_params`; with those in place, every case listed above is handled correctly.

## 3.9 Tests



Figure 3.1: Full ingestion pipeline output against the real Petstore documentation fixture: 19 of 19 endpoints detected

```
1 import json
2 from pathlib import Path
3
4 import pytest
5 from spekforge.ingest.chunking import split_into_chunks
6 from spekforge.ingest.markitdown_ingestor import MarkdownIngestor
7 from spekforge.ingest.router import route
8 from spekforge.ingest.structured_ingestor import StructuredIngestor
9 from spekforge.models.chunk import SourceType
10
11 def test_split_into_chunks_detects_endpoints_by_heading():
12     markdown = """## GET /pets/{id}
13 Retrieve a pet by id.
14
15 ## POST /pets
16 Create a new pet.
17 """
18     chunks = split_into_chunks(markdown, "petstore.md")
19
20     assert len(chunks) == 2
21     assert chunks[0].candidate_method == "GET"
22     assert chunks[0].candidate_path == "/pets/{id}"
```

```
23     assert chunks[1].candidate_method == "POST"
24     assert chunks[1].candidate_path == "/pets"
25
26 def test_split_into_chunks_ignores_inline_mentions():
27     markdown = "You can call GET /pets/{id} to retrieve a pet"
28     chunks = split_into_chunks(markdown, "petstore.md")
29
30     assert chunks == []
31
32 def test_structured_ingestor_rejects_generic_json(tmp_path):
33     source = tmp_path / "not_structured.json"
34     source.write_text(json.dumps({"hello": "world"}))
35
36     assert StructuredIngestor().accepts(source) is False
```

Listing 3.10: tests/unit/test\_ingest.py

In total, nine tests cover this phase’s contract, including heading-boundary detection and its negative case, extension acceptance, all three `StructuredIngestor` formats including HAR type inference, generic JSON rejection, and the router’s dispatch and error paths. Together with the edge-case review, this test suite is what gives confidence that the ingestion of real documentation behaves as intended.